

Q5 Evaluation: Correctness Verification

5(a): OLS Solution via Pseudoinverse

Claude's answer:

```
w = np.linalg.pinv(x) @ y
Result: w = [1.0891, 0.1089]
```

✓ **CORRECT.** Verification: The pseudoinverse formula for underdetermined systems is $w = X^T(XX^T)^{-1}y$. With $X=[10,1]$, $XX^T=101$, so $w = [10/101, 1/101] \times 11 = [110/101, 11/101] = [1.0891, 0.1089]$. Claude's computation matches.

Verification check: $10 \times 1.0891 + 1 \times 0.1089 = 10.891 + 0.1089 = 11.0$ ✓
The solution satisfies the constraint exactly.

Note on the 10:1 ratio explanation: Claude claims 'minimum-norm distributes work proportionally to coefficients.' This is imprecise—minimum-norm means $\|w\|$ is minimized, which happens to produce this ratio because the gradient direction is $[10,1]$ (row of X). The claim about 'gradient with respect to w is $10\times$ larger' is correct for GD but conflates two different derivations (pseudoinverse vs GD).

5(b): Gradient Descent from Zero Init

Claude's answer:

```
net = torch.nn.Linear(2, 1, bias=False)
Initialize to zero, train with SGD
Result: w ≈ [1.0891, 0.1089]
```

✓ **CORRECT.** The claim that GD from zero converges to minimum-norm solution is a known result. The gradient $\nabla L = 2X^T(Xw-y)$ lies in $\text{row}(X)$, so w stays in $\text{row}(X)$ throughout training. The minimum-norm solution is the unique solution in $\text{row}(X)$.

Technical precision: Claude's explanation is correct. The statement 'gradient always lies in row space of X ' is the key insight. This is why implicit regularization works—you can't leave the subspace you start in.

5(c): Dropout Expanded Dataset

Claude's answer:

```
X_dropout = [[0,0], [0,2], [20,0], [20,2]]
y = [11, 11, 11, 11]
Result: w = [0.367, 3.667]
```

✓ **CORRECT.** The expanded dataset correctly enumerates all $2^2=4$ dropout masks. Kept features are scaled by $1/(1-0.5)=2$. The OLS solution on this expanded dataset gives $[0.367, 3.667]$.

Verification: Can verify via pinv calculation on the 4×2 matrix. The ratio inversion (from 10:1 to 1:10) is correctly observed.

■ **MINOR ISSUE in verification:** Claude writes ' $10 \times 0.367 + 3.667 = 7.33 \neq 11$ ' and says this doesn't satisfy the original equation. This is TRUE but the explanation is incomplete—it's not just 'optimizing expected loss,' it's that the $[0,0]$ row contributes to the OLS objective, pulling the solution away from exact satisfaction.

5(d): Dropout Layer Implementation

Claude's answer:

```
mask = (torch.rand_like(x) > self.p).float()
return x * mask / (1 - self.p)
```

✓ **CORRECT.** This is standard inverted dropout. The mask uses > (not >=), which is fine since $P(\text{rand}=p)=0$ for continuous uniform. Scaling by $1/(1-p)$ maintains $E[\text{output}]=\text{input}$.

Verification: $E[\text{mask}] = (1-p)$, so $E[x * \text{mask} / (1-p)] = x * (1-p) / (1-p) = x$ ✓

Training curve explanation: Claude correctly identifies high variance due to `batch_size=1` and different dropout masks per iteration. The claim that 'loss = 121' when both dropped is correct: $(0-11)^2=121$.

5(e): Large Batch Behavior

Claude's answer:

Batch gradient = $(1/1024) \sum \nabla L$

By Law of Large Numbers, variance reduced by $\sim 1024\times$

✓ **CORRECT.** $\text{Var}(\text{sample mean}) = \text{Var}(\text{individual})/n$. With $n=1024$, variance drops by $1024\times$, std dev drops by $\sim 32\times$. This explains the smooth training curve.

5(f): Weight and Contribution Analysis

Claude's answer:

Without dropout: $10 \times w_{\text{cheat}}$ contributes 10.89, w_{clean} contributes 0.109 (ratio 100:1)

With dropout: $10 \times w_{\text{cheat}}$ contributes 3.67, w_{clean} contributes 3.667 (ratio 1:1)

✓ **CORRECT.** The contribution analysis is accurate:

- No dropout: $10 \times 1.089 = 10.89$ vs 0.109 $\rightarrow$ 100:1 ratio
- Dropout: $10 \times 0.367 = 3.67$ vs 3.667 $\rightarrow$ 1:1 ratio

Dropout equalizes feature contributions.

'Eggs in basket' intuition: Claude's explanation is qualitatively correct—the network can't rely on one feature because it might be dropped. The result is more balanced weights.

5(i): CIFAR-10 with Cheating Feature

Claude's answer:

No dropout: $\sim 99\%$ with cheat, $\sim 25\text{-}30\%$ clean

With dropout ($p=0.75$): $\sim 55\text{-}60\%$ with cheat, $\sim 40\text{-}45\%$ clean

✓ **CORRECT interpretation.** The key insight is correct: dropout prevents shortcut learning by forcing the network to use real image features as backup when cheating pixels are dropped.

Note: Exact numbers depend on the specific run, but the qualitative pattern (dropout improves clean accuracy at cost of cheat accuracy) is the expected result.

Summary: Q5 Correctness Assessment

Part	Code	Math	Explanation	Issues Found
5(a)	✓	✓	✓	Minor: conflates pinv/GD derivations
5(b)	✓	✓	✓	None
5(c)	✓	✓	~	Incomplete explanation of why solution $\neq 11$
5(d)	✓	✓	✓	None
5(e)	N/A	✓	✓	None
5(f)	N/A	✓	✓	None
5(i)	N/A	N/A	✓	None (results are run-dependent)

Overall Assessment:

All code implementations are correct. All numerical results are verifiable. Minor issues are in explanations rather than solutions—Claude occasionally conflates different mathematical derivations that happen to give the same answer (e.g., pseudoinverse geometry vs gradient descent dynamics). These are pedantic concerns; the answers would receive full credit.

Error count: 0 incorrect answers, 2 imprecise explanations